

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Информационных технологий и управления
Кафедра Интеллектуальных информационных технологий

ОТЧЁТ
по ознакомительной практике

Выполнил:

А. Д. Аникеев

Студент группы
421701

Проверил:

Н. В. Малиновская

Минск 2025

СОДЕРЖАНИЕ

Введение	3
1 Формализация и сравнительный анализ языков представления информации в интеллектуальных системах: Flora-2 и PowerLoom . . .	4
2 Формализация и сравнительный анализ инструментов для разработки интеллектуальных систем: MadKit и Ray	9
Заключение	14
Список использованных источников	15

ВВЕДЕНИЕ

Цель:

Провести сравнительный анализ современных технологий разработки интеллектуальных систем, закрепить навыки формализации информации, а также приобрести и развить навыки исследования, анализа и сравнения технологических решений.

Задачи:

1. Изучить языки логического программирования (PowerLoom и Flora-2) как инструменты формализации декларативных знаний, провести анализ их особенностей, выявить сходства и различия между ними.
2. Рассмотреть мультиагентные платформы и фреймворки распределённых вычислений (MadKit и Ray), выявить их функциональные возможности и подходы к представлению знаний, сходства и отличительные особенности.
3. Формализовать полученные результаты в виде SCn-кода.
4. Оформить библиографические источники к полученным результатам по ГОСТ 7.1-2003.

1 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ЯЗЫКОВ ПРЕДСТАВЛЕНИЯ ИНФОРМАЦИИ В ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМАХ: FLORA-2 И POWERLOOM

Flora-2

- :=** [Объектно-ориентированный язык логического программирования, расширение F-logic]
- :=** [Flora-2 (Flora with Objects)]
- ∈** *логический язык*
- ⇒** *разработчик**:
 - Майкл Кифер
 - Гуйзхенг Ян
 - Другие участники проекта Flora
- ⇒** *год создания**:
1998
- ⇒** *основная парадигма**:
объектно-ориентированное логическое программирование
- ⇒** *описание**:
[Flora-2 — современный язык логического программирования, сочетающий F-logic, HiLog и транзакционную логику. Расширяет Prolog, добавляя объектно-ориентированные возможности, модульность и поддержку семантических веб-технологий.]
- ⇒** *описание**:
[Flora-2 обеспечивает строгую типизацию, поддержку метапрограммирования, обработку онтологий и работу с XML/RDF. Язык особенно эффективен для задач интеграции данных, веб-семантики и сложных систем представления знаний.]
- ⇒** *назначение**:
[Применяется для разработки интеллектуальных систем, интеграции разнородных данных, создания онтологий, семантических веб-приложений и сложных экспертных систем.]
- ⇒** *основные конструкции**:
 - {• [объекты с методами и наследованием]
 - [модули и пакеты]
 - [транзакционная логика]
 - [метапрограммирование через HiLog]
 - }
- ⇒** *стратегия вывода**:
top-down с табулированием (XSB Prolog engine)
- ⇒** *поддержка**:
 - [объектно-ориентированное программирование]
 - [семантические веб-технологии]
 - [XML/RDF интеграция]
 - [транзакционная логика]
- ⇒** *применение**:
 - *семантический веб*
 - *интеграция данных*
 - *онтологическое моделирование*

- *сложные экспертные системы*
 - *анализ социальных сетей*
 - *обработка метаданных*
 - *логический вывод в гетерогенных средах*
- ⇒ *примеры использования**:
- {• [Разработка онтологий для семантического веба]
 - [Интеграция разнородных баз данных и веб-сервисов]
 - [Системы управления знаниями в корпоративных средах]
 - [Анализ социальных графов и сетевых структур]
 - [Моделирование бизнес-правил и регуляторных требований]
 - [Создание интеллектуальных агентов для семантических веб-сервисов]
 - [Разработка сложных экспертных систем с объектной организацией знаний]
- ⇒ *реализации**:
- {• *Flora-2 на XSB Prolog*
 - *Flora-2 с поддержкой RDF/OWL*
- ⇒ *особенности**:
- {• [комбинация F-logic, HiLog и транзакционной логики]
 - [поддержка семантических веб-стандартов]
 - [объектно-ориентированное моделирование]
 - [модульность и пакетная организация кода]
 - [интеграция с XML/RDF]
 - [поддержка метапрограммирования]
- ⇒ *ограничения**:
- {• [более сложный синтаксис по сравнению с классическим Prolog]
 - [меньшее распространение, чем у Prolog]
 - [требует понимания объектно-ориентированных концепций]
- ⇒ *библиографические источники**:
- {• [Yang G., Kifer M. Flora-2: A Rule-Based Knowledge Representation and Inference Infrastructure for the Semantic Web // ODBASE. – 2003.]
 - [Kifer M. Rule-Based Programming in the Semantic Web // ACM SIGMOD Record. – 2004.]
 - [Kifer M., Lausen G., Wu J. Logical foundations of object-oriented and frame-based languages // JACM. – 1995.]
 - [Документация Flora-2. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <http://flora.sourceforge.net/docs.php>]

PowerLoom

- := [Язык логического представления знаний, разработанный в ISI]
- := [PowerLoom Knowledge Representation System]
- ∈ *логический язык*
- ∈ *система представления знаний*
- ⇒ *разработчик**:
- *Институт информатики USC (ISI)*
 - *Роберт Макгрегор*
- ⇒ *год создания**:

- 1990-е
- ⇒ *основная парадигма**:
декларативное программирование
 - ⇒ *основная парадигма**:
логическое представление знаний
 - ⇒ *описание**:
[PowerLoom — мощная система представления знаний и логического вывода, основанная на языке STELLA (Strongly Typed English-Like Logic Language). Сочетает возможности дедуктивных баз данных, логического программирования и онтологических моделей.]
 - ⇒ *описание**:
[PowerLoom поддерживает дескриптивную логику, правила, модульность и объектно-ориентированные конструкции. Система оптимизирована для сложных задач логического вывода и интеграции знаний из различных источников.]
 - ⇒ *назначение**:
[Используется для построения интеллектуальных систем, требующих сложного логического вывода, интеграции знаний и работы с онтологиями. Применяется в экспертных системах, семантических технологиях и аналитических приложениях.]
 - ⇒ *основные конструкции**:
 - { • [декларации концептов]
 - [правила и ограничения]
 - [модули и контексты]
 - [запросы и дедукции]
 - }
 - ⇒ *стратегия вывода**:
гибридная (*forward и backward chaining*)
 - ⇒ *поддержка**:
 - [дескриптивная логика]
 - [модульность]
 - [объектно-ориентированные конструкции]
 - [сложные правила вывода]
 - ⇒ *применение**:
 - экспертные системы
 - семантические технологии
 - интеграция знаний
 - аналитические системы
 - логический вывод
 - онтологическое моделирование
 - ⇒ *примеры использования**:
 - { • [Построение сложных экспертных систем для медицины и инженерии]
 - [Интеграция данных из разнородных источников знаний]
 - [Семантический анализ текстовой информации]
 - [Моделирование бизнес-правил и регуляторных требований]
 - [Разработка интеллектуальных помощников принятия решений]
 - [Создание онтологий предметных областей]
 - }
 - ⇒ *реализации**:
 - { • PowerLoom (оригинальная версия)
 - STELLA (базовый язык)
 - PL-Cloud (облачная версия)

- ⇒ }
 - ⇒ особенности*:
 - { • [поддержка дескриптивной логики]
 - [модульная архитектура]
 - [гибридный механизм вывода]
 - [интеграция с языком STELLA]
 - [поддержка сложных онтологий]
 - ⇒ }
 - ⇒ ограничения*:
 - { • [кривая обучения для сложных конструкций]
 - [меньшее распространение по сравнению с Prolog]
 - [требуется понимания логического моделирования]
 - ⇒ }
 - ⇒ библиографические источники*:
 - { • [McGregor R. The PowerLoom Knowledge Representation and Reasoning System // SRI International Technical Report, 1991.]
 - [PowerLoom Reference Manual // USC Information Sciences Institute, 2020.]
 - [Kifer M., Lausen G., Wu J. Logical foundations of object-oriented and frame-based languages // JACM, 1995.]
 - [Документация PowerLoom. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <http://www.isi.edu/isd/LOOM/PowerLoom/>]
 - ⇒ }

Сравнение языков логического программирования *Flora-2* и *PowerLoom*

- = { • *Flora-2*
- *PowerLoom*
- }
- ⇒ сходства*:
 - { • [Оба являются системами представления знаний на основе логического программирования]
 - [Поддерживают объектно-ориентированный подход к моделированию знаний]
 - [Обеспечивают сложный логический вывод и работу с онтологиями]
 - [Имеют модульную архитектуру и поддержку контекстов]
 - [Применяются для построения экспертных систем и интеллектуальных приложений]
 - [Разработаны для интеграции разнородных источников знаний]
- ⇒ различия*:
 - { • [Flora-2 основан на F-logic и HiLog, PowerLoom – на языке STELLA]
 - [Flora-2 использует XSB Prolog как движок, PowerLoom имеет собственную реализацию]
 - [Flora-2 ориентирован на семантический веб и XML/RDF, PowerLoom – на общие задачи представления знаний]
 - [PowerLoom поддерживает дескриптивную логику, Flora-2 делает акцент на транзакционной логике]
 - [Flora-2 имеет более выраженную ориентацию на распределенные вычисления]
 - [PowerLoom предоставляет более развитые средства для объяснения логического вывода]

- [Flora-2 лучше подходит для веб-ориентированных приложений, PowerLoom – для автономных экспертных систем]
- }
⇒ *рекомендации по выбору**:
- {• [Flora-2 рекомендуется для семантических веб-приложений, интеграции с RDF/OWL и распределенных систем знаний]
 - [Flora-2 предпочтителен при необходимости работы с XML-данными и веб-сервисами]
 - [PowerLoom следует выбирать для традиционных экспертных систем с требованием объяснимого вывода]
 - [PowerLoom лучше подходит для сложного онтологического моделирования в замкнутых системах]
 - [Для образовательных целей PowerLoom может быть предпочтительнее благодаря более простой архитектуре]
 - [Flora-2 рекомендуется при необходимости интеграции с существующими Prolog-системами]
- }

2 ФОРМАЛИЗАЦИЯ И СРАВНИТЕЛЬНЫЙ АНАЛИЗ ИНСТРУМЕНТОВ ДЛЯ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ: MADKIT И RAY

MadKit

- := [Мультиагентная платформа для разработки распределенных интеллектуальных систем]
- := [Multi-agent Development Kit]
- ∈ *инструмент для разработки мультиагентных систем*
 - := [платформа для моделирования агентов]
- ∈ *платформа искусственного интеллекта*
- ⇒ *разработчик**:
Университет Пьера и Марии Кюри (Париж VI)
- ⇒ *год создания**:
1998
- ⇒ *текущая стабильная версия**:
[5.2]
- ⇒ *дата выпуска текущей версии**:
[15 сентября 2023]
- ⇒ *лицензия**:
LGPL
- ⇒ *язык разработки**:
Java
- ⇒ *назначение**:
[Создание и управление распределенными мультиагентными системами для решения сложных задач координации и принятия решений]
- ⇒ *варианты исполнения**:
 - { • *MadKit Desktop*
 - := [Настольная версия с графическим окружением]
 - ⇒ *примечание**:
[Включает агентную консоль, мониторинг и визуализацию агентных взаимодействий]
 - *MadKit Kernel*
 - := [Минимальное ядро для встраивания в приложения]
 - ⇒ *примечание**:
[Размер менее 1MB, поддерживает headless-режим работы]
- }
 - ⇒ *функциональные возможности**:
 - { • [моделирование агентов]
 - := [создание и управление агентами и их сообществами]
 - ⇒ *пример использования**:
[Моделирование транспортных потоков с агентами-автомобилями]
 - [организация агентов]
 - ⇒ *принципы организации**:
 - { • [ролевые модели]
 - [иерархические структуры]
 - [группы и подгруппы]
 - }

- [коммуникация агентов]
 - ⇒ *механизмы коммуникации**:
 - [обмен сообщениями ACL]
 - [черные доски]
 - [событийная модель]
 - [визуализация]
 - := [графическое представление агентных взаимодействий]
 - ⇒ *пример использования**:
 - [Визуализация динамики рынка в экономических симуляциях]
 - [поддержка плагинов]
 - ⇒ *категории плагинов**:
 - [плагины поведения агентов]
 - [плагины коммуникации]
 - [плагины визуализации]
- ⇒ *архитектурные особенности**:
- Микроядро
 - Модульность
 - Поддержка распределенных вычислений
 - Гибкая модель ролей
- ⇒ *преимущества**:
- [Легковесность и простота использования]
 - [Гибкая модель организации агентов]
 - [Открытая архитектура]
 - [Хорошая документация и примеры]
 - [Поддержка распределенных сценариев]
- ⇒ *недостатки**:
- [Ограниченная поддержка промышленных стандартов FIPA]
 - [Слабая интеграция с современными AI-фреймворками]
 - [Меньшее сообщество по сравнению с JADE]
- ⇒ *библиографические источники**:
- [Gutknecht O., Ferber J. The MadKit agent platform architecture // International Workshop on Agent-Oriented Software Engineering. 2000.]
 - [Ferber J., Gutknecht O. A meta-model for the analysis and design of organizations in multi-agent systems // Proceedings of ICMS. 1998.]
 - [Official MadKit Documentation [Электронный ресурс] // URL: <http://www.madkit.net> (дата обращения: 12.05.2025).]
 - [Multi-Agent Systems: Simulation and Applications [Электронный ресурс] // CRC Press, 2024.]

Ray

- := [Распределенная вычислительная платформа для масштабируемого машинного обучения и обработки данных]
- := [Ray: A Distributed Execution Framework]

- ∈ фреймворк для распределенных вычислений
- ∈ платформа для искусственного интеллекта
- ⇒ разработчик*:
Anyscale
- ⇒ основатели*:
Роберт Ниш, Ион Штойка, Филипп Мориц
- ⇒ год создания*:
2017
- ⇒ текущая версия*:
[2.10.0]
- ⇒ дата выпуска*:
[Май 2024]
- ⇒ лицензия*:
Apache 2.0
- ⇒ язык разработки*:
Python, C++
- ⇒ назначение*:
[Упрощение разработки и развертывания распределенных приложений для машинного обучения и обработки данных]
- ⇒ ключевые компоненты*:
 - { • Ray Core
 - ⇒ описание*:
[Базовый API для распределенного выполнения задач и состояний]
 - Ray AI Runtime (AIR)
 - ⇒ описание*:
[Библиотеки для сквозного ML pipeline: данные, обучение, тюнинг, инференс]
 - Ray Serve
 - ⇒ описание*:
[Масштабируемый сервис для развертывания ML моделей]
 - Ray Tune
 - ⇒ описание*:
[Библиотека для гиперпараметрического тюнинга]
 - Ray RLlib
 - ⇒ описание*:
[Масштабируемая библиотека для обучения с подкреплением]
- ⇒ архитектурные особенности*:
 - { • децентрализованная архитектура
 - глобальный планировщик задач
 - распределенное хранилище объектов
 - горизонтальное масштабирование
 - поддержка гетерогенных кластеров
- ⇒ функциональные возможности*:
 - { • [Распределенное выполнение Python-функций]
 - [Параллельная обработка данных]
 - [Оркестрация распределенных состояний]
 -

- [Автомасштабирование ресурсов]
- [Интеграция с ML/DL фреймворками]
- ⇒ *примеры**:
 - [TensorFlow, PyTorch, XGBoost, Scikit-learn]
 - [Поддержка Kubernetes и облачных платформ]
 - [Интерактивная отладка и профилирование]
- }
- ⇒ *преимущества**:
 - {• [простота перехода от локального к распределенному коду]
 - [гибкость для различных ML/DL задач]
 - [автоматическое управление ресурсами]
 - [высокая производительность]
 - [активное сообщество и поддержка]
 - [интеграция с экосистемой Python]
- }
- ⇒ *недостатки**:
 - {• [ограниченная поддержка языков кроме Python]
 - [кривая обучения для сложных распределенных сценариев]
 - [требуется настройка для оптимальной производительности]
- }
- ⇒ *примеры использования**:
 - {• [Обучение крупных ML моделей]
 - [Гиперпараметрический поиск]
 - [Масштабируемый инференс моделей]
 - [Обработка больших данных]
 - [Распределенное обучение с подкреплением]
 - [Параллельные симуляции]
- }
- ⇒ *библиографические источники**:
 - {• [Moritz P. et al. Ray: A Distributed Framework for Emerging AI Applications // 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). 2018.]
 - [Official Ray Documentation [Электронный ресурс] // URL: <https://docs.ray.io> (дата обращения: 12.05.2025).]
 - [Ray: A Distributed Execution Framework for Reinforcement Learning Applications [Электронный ресурс] // URL: <https://arxiv.org/abs/1712.05889> (дата обращения: 12.05.2025).]
 - [Scaling Python for Machine Learning with Ray [Электронный ресурс] // O'Reilly, 2023.]
- }

Сравнение инструментов для разработки интеллектуальных систем MadKit и Ray

- = { • *MadKit*
- *Ray*
- }
- ⇒ *сходства**:
- {• [Оба являются платформами для разработки распределенных систем]
 - [Поддерживают параллельные вычисления]
 - [Имеют открытый исходный код]
 - [Используются в задачах искусственного интеллекта]

- [Обеспечивают масштабируемость решений]
- [Поддерживают работу в гетерогенных средах]
- [Имеют активные сообщества пользователей]

⇒

различия:*

- [MadKit – мультиагентная платформа, Ray – фреймворк для распределенных вычислений]
- [MadKit использует агентно-ориентированную парадигму, Ray – парадигму задач и акторов]
- [MadKit ориентирован на моделирование сложных взаимодействий, Ray – на масштабирование ML-вычислений]
- [MadKit написан на Java, Ray – на Python и C++]
- [MadKit имеет встроенные механизмы организации агентов, Ray предоставляет низкоуровневые примитивы]
- [Ray имеет специализированные библиотеки для ML (RLlib, Tune), MadKit – более общую архитектуру]
- [Ray лучше интегрирован с современными ML-фреймворками]
- [MadKit предоставляет больше возможностей для моделирования социальных взаимодействий]
- [Ray имеет более развитую систему автоматического масштабирования]

⇒

рекомендации по выбору:*

- [MadKit рекомендуется для моделирования сложных социальных систем и мультиагентных взаимодействий]
- [MadKit подходит для задач, где важна организация и координация автономных агентов]
- [MadKit хороший выбор для образовательных целей в области мультиагентных систем]
- [Ray целесообразен для масштабирования машинного обучения и обработки данных]
- [Ray рекомендуется для распределенного обучения моделей и гиперпараметрического поиска]
- [Ray оптимален для проектов, требующих интеграции с популярными ML-фреймворками]
- [Для сложных распределенных ML-пайплайнов предпочтительнее Ray]
- [Для моделирования социальных и экономических систем лучше подходит MadKit]

ЗАКЛЮЧЕНИЕ

В ходе проведённого исследования был осуществлён сравнительный анализ современных технологий разработки интеллектуальных систем, включая языки представления знаний и платформы для распределённых вычислений.

В первом разделе работы были подробно рассмотрены языки представления знаний Flora-2 и PowerLoom. Проанализированы их концептуальные основы и архитектурные особенности, выявлены общие черты и различия в подходах к логическому программированию и организации знаний. Особое внимание уделено возможностям интеграции с семантическими веб-технологиями и поддержке различных парадигм логического вывода. Результаты анализа были формализованы с помощью SCn-кода.

Во втором разделе исследованы современные платформы для распределённых вычислений Ray и MadKit. Проведен анализ их функциональных возможностей, механизмов масштабирования и подходов к организации параллельных вычислений. Рассмотрены особенности архитектуры каждой платформы и их применение в задачах искусственного интеллекта. Информация о каждом инструменте также была формализована на SCn-коде.

Результаты работы могут быть использованы при проектировании комплексных интеллектуальных систем, сочетающих возможности логического вывода и распределённой обработки данных, а также при выборе технологических решений для конкретных задач в области искусственного интеллекта и обработки знаний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Yang, G. Flora-2: A rule-based knowledge representation and inference infrastructure for the semantic web / G. Yang, M. Kifer // ODBASE. — 2003.
- [2] Kifer, M. Rule-based programming in the semantic web / M. Kifer // ACM SIGMOD Record. — 2004.
- [3] Kifer, M. Logical foundations of object-oriented and frame-based languages / M. Kifer, G. Lausen, J. Wu // JACM. — 1995.
- [4] Документация flora-2. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <http://flora.sourceforge.net/docs.php>.
- [5] McGregor, R. The powerloom knowledge representation and reasoning system: Technical report / R. McGregor: SRI International, 1991.
- [6] USC Information Sciences Institute. — PowerLoom Reference Manual, 2020.
- [7] Документация powerloom. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <http://www.isi.edu/isd/LOOM/PowerLoom/>.
- [8] Gutknecht, O. The madkit agent platform architecture / O. Gutknecht, J. Ferber // International Workshop on Agent-Oriented Software Engineering. — 2000.
- [9] Ferber, J. A meta-model for the analysis and design of organizations in multi-agent systems / J. Ferber, O. Gutknecht // Proceedings of ICMS. — 1998.
- [10] Official madkit documentation. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <http://www.madkit.net>.
- [11] Multi-Agent Systems: Simulation and Applications. — CRC Press, 2024.
- [12] Ray: A distributed framework for emerging ai applications / P. Moritz [et al.] // 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI). — 2018.
- [13] Official ray documentation. — 2025. — Электронный ресурс. Дата обращения: 12.05.2025. <https://docs.ray.io>.
- [14] Ray: A distributed execution framework for reinforcement learning applications / R. Nishihara [et al.] // arXiv preprint arXiv:1712.05889. — 2018. — Электронный ресурс. Дата обращения: 12.05.2025.
- [15] Scaling Python for Machine Learning with Ray. — O'Reilly, 2023.